

loaderEFD: Sistema de *ETL* para Escrituração Fiscal Digital baseado em regras declarativas

Relatório técnico

Michel Diniz Medeiros

Agosto de 2026

Centro de Informática – Universidade Federal da Paraíba (UFPB)

Orientador: Prof. Dr. Marcelo Iury de Sousa Oliveira



1. Introdução
2. Criação do Documento de Metadados
3. Metodologia
4. Resultados e Análises
5. Conclusões e Trabalhos Futuros

Introdução

O SPED e a Escrituração Fiscal Digital de ICMS e IPI

- **O SPED:** É um sistema que busca modernizar a interação entre contribuintes e o fisco através da informatização das obrigações acessórias.
- **O EFD ICMS IPI:** É de uso obrigatório para os contribuintes de ICMS e IPI e contém registros de apuração de impostos, documentos fiscais e outras informações de interesse.
- **A urgência legal:** Prazo de 5 anos para auditorias antes que seja extinto o crédito tributário.

Com isso:

- **Auditorias:** É de interesse do Fisco realizar auditorias sobre esses documentos em tempo hábil.
- **Empecilhos:** Arquivos em texto plano hierárquico e atualizações constantes de leiaute pelo Fisco.
- **O desafio gerado:** Alto custo de manutenção e quebra de sistemas com código engessado.

Portanto, nossa equipe definiu como objetivo geral:

- Desenvolver um *pipeline* ETL dinâmico e orientado a metadados para EFD ICMS IPI.
- Realizar a substituição de regras no código-fonte por contratos em formato **JSON**.
- Viabilizar a ingestão escalável dos documentos, com um processamento resiliente e genérico, junto com armazenamento relacional preciso com uso do *PostgreSQL*.

Criação do Documento de Metadados

Listagem 1: Exemplo de documento EFD

```
|0000|EMP|0|01012024|31012024|EMPRESA TESTE LTDA|12345678901234|12345678901|EM  
|EMPRESA TESTE |EMPRESA|EMPRESA TESTE LTDA|EMPRESA T|E|1|^CRLF  
|0001|0|^CRLF  
|0005|EMPRESA TESTE LTDA|EMPRESA |COMERCIAL ABC|EMPRESA TE|FULANO DE TAL|RUA  
DAS FLORES 123|EMPRESA TES|COMERCIAL A|COMERCIAL ABC|^CRLF  
|0100|COMERCIAL ABC|12345678901|EMPRESA TESTE L|12345678901234|COMERCIA|  
contribuinte|COMERCIAL |12345678901234567890|MARIA SANTOS|FULANO DE T|RUA  
DAS FLO|FULANO DE TAL|COMERCI|^CRLF  
|0990|6|^CRLF  
|B001|0|^CRLF  
|B020|99|100|SAO PAULO|CO|FU|COM|COMERCIAL|EMPRESA TESTE LTDA|01042024|FULANO  
|0,00|1,00|99,99|100,00|99,99|100,00|1234,11|0,01|999,99|PRODUTO A1|^CRLF
```

Fonte: Elaborado pelo autor (2026)

Essa sintaxe possui alguns empecilhos que o presente trabalho necessita lidar:

- Ausência de uma representação oficial que seja legível por máquina.
- Estrutura totalmente implícita.

A equipe então decidiu:

- Fazer a tradução da representação oficial para JSON.
- Determinar otimizações prévias que já poderiam ser realizadas para evitar custos de leitura.

Listagem 2: Modelo JSON de Regras de Metadados para a EFD ICMS IPI

```
{
  "nome_tabela": "",
  "registro_principal": "",
  "registros": [{
    "caminho": "",
    "ocorrencia": "",
    "campos": [{
      "nome_campo": "",
      "tipo_dado": ""
    }]
  }],
  ...
}
```

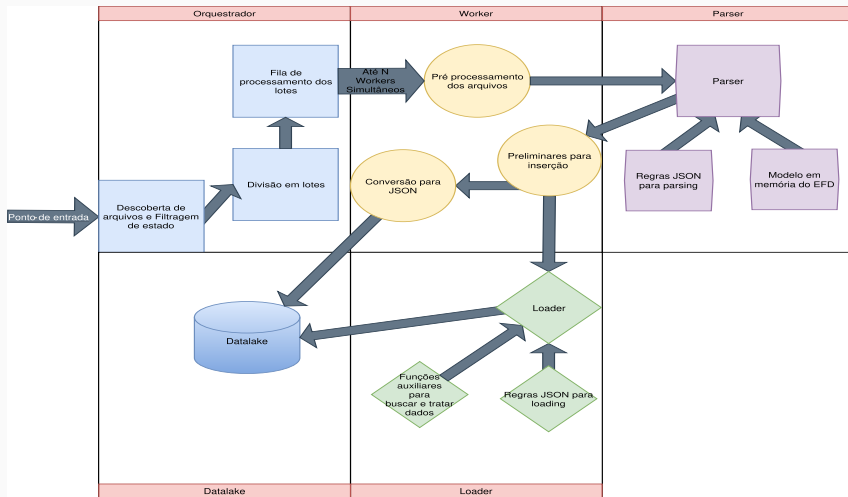
JSON de Regras de Metadados

```
...  
"colunas": [{  
  "nome_coluna": "",  
  "tipo_dado": "",  
  "origem": "",  
  "extras": {  
    "caminho": "",  
    "nome_campo": ""  
  }  
}],  
"chave_primaria": []  
}
```

Fonte: Elaborado pelo autor (2026)

Metodologia

Figura 1: Diagrama de arquitetura de ETL para arquivos EFD ICMS IPI.



Fonte: Elaborado pelo autor (2026).

Responsável pela conversão do formato textual para uma representação intermediária:

- Funciona por meio varredura sequencial, iterando linha a linha sobre o documento.
- Utiliza descida recursiva para fazer a inserção dos registros na estrutura em memória.
- Garante validade de cardinalidade e da quantia dos campos.

Responsável por orquestrar a inserção da representação intermediária no banco de dados:

- **Varredura em profundidade:** A representação intermediária é varrida usando um algoritmo **DFS** recursivo.
- **Uso de *dispatcher*:** Ao encontrar um registro, se utiliza do campo “colunas” dos metadados para fazer o despache dinâmico do método que extrai as informações daquela coluna.
- **Integridade primeiro:** Foco na consistência em detrimento da tolerância a falhas.

Resultados e Análises

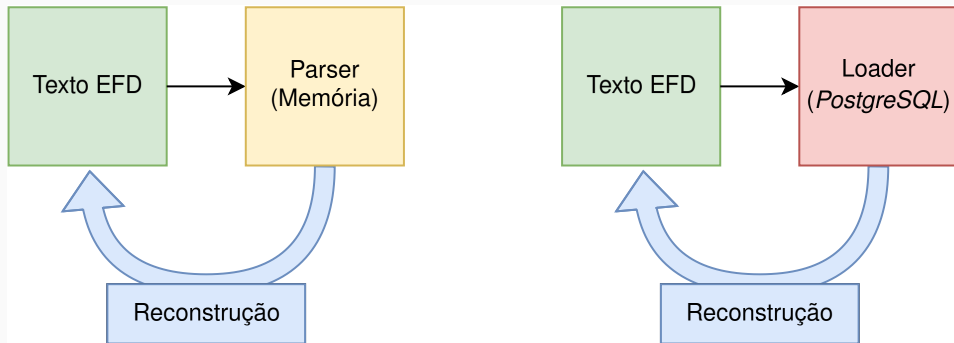
Quadro 1: Especificações do ambiente computacional

Componente	Especificação Técnica
Infraestrutura	Máquina Virtual sob <i>hypervisor</i> VMware (<i>virtualização total</i>)
Processador	Intel® Xeon® Gold 5320 @ 2.20GHz
Topologia da CPU	16 núcleos lógicos (16 <i>sockets</i> , 1 <i>core/socket</i> , 1 <i>thread/core</i>)
Cache	L1: 80 KB / L2: 1,25 MB / L3: 39,9 MB (Compartilhado)
Memória RAM	93Gi, com 4Gi de swap
Armazenamento (Raiz do sistema)	SSD de 135GB
Armazenamento (Volume dedicado ao banco de dados)	SSD de 36TB

Fonte: Elaborado pelo autor (2026).

Foi seguida uma metodologia preliminar baseada em *round-trip* para testar a integridade:

Figura 2: Metodologia *round-trip*



Fonte: Elaborado pelo autor (2026).

Já para verificar a qualidade do tratamento de erros:

- Geração de arquivos com falhas pré-determinadas.
- Injeção de erros durante o processamento.

Nos testes em ambiente de homologação, obtemos:

- **Volume Processado:** 637.361 arquivos.
- **Anomalias Isoladas:** 15 arquivos.
- **Desempenho Unitário:** Cerca de 90% dos *Workers* processam um lote de 100 arquivos em menos de 20 minutos.
- **Outliers:** Algumas instâncias de *Workers* demoraram até 2 horas para finalizar.

Os testes em homologação revelaram algumas limitações:

- **Limite físico do *PostgreSQL*:** Limite de 256MB para campos únicos de um JSONB.
- **Degradação de E/S:** Queda de vazão em inserções massivas sob transação única devido ao motor de consistência do *SGBD*.
- **Conflitos Normativos:** Divergências entre o Guia Prático e a Nota Técnica para o registro H020.

Conclusões e Trabalhos Futuros

No fim, nossa equipe conseguiu:

- Desenvolver um *ETL* dinâmico validado para arquivos EFD ICMS IPI.
- Executar com sucesso o desacoplamento entre lógica de código e regras de negócio por meio de contratos em JSON.
- Garantir a integridade do *pipeline* por meio de *round-trip*.
- Descobrir limitações no modelo atual que podem ser usadas para refinar este ou outros projetos de ETL.

Delimitamos como possíveis trabalhos futuros:

- **Otimização de Carga de Dados:** Uso de *staging tables* e *bulk inserts* para resolver o gargalo encontrado.
- **Testes de Estresse e Escala:** Realizar experimentos quantitativos em ambientes de carga extrema.
- **Expansão do Modelo:** Aplicação da arquitetura modelada neste trabalho para outros documentos textuais complexos e voláteis.

Obrigado!

Dúvidas?